Software Proposal: Bus Ticket Consolidation & Inventory Management System (Web or Windows Application)

Executive Summary

This proposal outlines the development of a **combined Web or Windows Application** for ALPS The Bus Inc. that integrates two critical modules:

1. **Bus Ticket Consolidation System** – centralizing ticket tracking, and reporting.
2. **Inventory Management System** – managing stock, supplies, and assets with real-time visibility.

By consolidating both systems into one platform, the company will reduce manual processes, improve accuracy, and streamline operations for both ticket and inventory tracking.

Project Objectives

- Develop a secure **internal web or windows application** with two main modules:
    - **Ticket Consolidation** – tracking, approvals, reporting.
    - **Inventory Management** – stock entry, tracking, reporting.
- Automate approval workflows for ticket consolidation and inventory requisitions.
- Provide centralized dashboards for administrators.
- Generate detailed **financial and operational reports**.
- Ensure scalability for future integration with other business systems.

Scope of Work

- **Requirement Analysis & Documentation** - Workshops with travel, HR, finance, and warehouse teams.
- **UI/UX Design** - Unified responsive design for both modules.
- **Core Features Development**
    - *Ticket Module*: ticket consolidation, approval workflow, reporting.
    - *Inventory Module*: Stock entry, issuance, tracking, low-stock alerts, reporting.
- **Admin Dashboard** - Consolidated reporting across both modules.
- **Testing & QA** - Functional, security, and performance testing.
- **Deployment & Training**
    - Deploy to company server or cloud.
    - Conduct training sessions for staff.

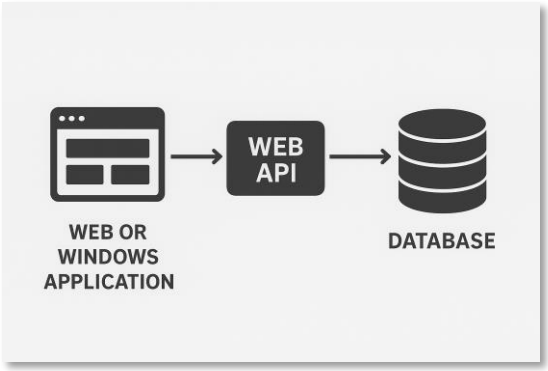
- **Maintenance & Support**
    - 2-month support for bug fixes and enhancements.

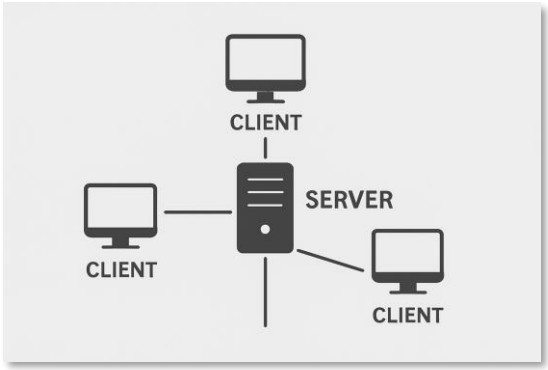
Technology Stack (Proposed)

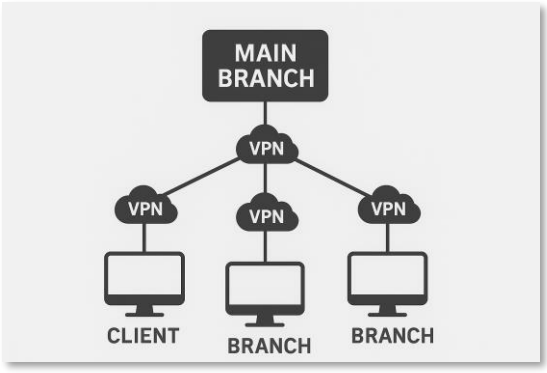
- **Frontend**: React.js or Windows Forms App
- **Backend**: Node.js (Express.js) or ASP.Net Core Web API
- **Database**: MongoDB (modules for tickets & inventory) or MSSQL
- **Authentication**: JWT with role-based access (employee, manager, admin, warehouse staff)
- **Infrastructure**: on-premises server with RDS (Disaster Recovery System)
- **Security**: SSL encryption, audit logging, access control

Infrastructure (Proposed)

- The whole system **resides on 1 server within main branch** with *Disaster Recovery System*
- The application will have the option to **connect to other client outside of the main branch** to sync data or do it **manually via a data file or encode entry**







Technology Advantages

Feature / Aspect	Web Applications 🌐	Windows/Desktop Applications 💻
Installation	No installation needed; runs in a browser	Requires installation on each device
Accessibility	Accessible anywhere with internet (*unless specifically designed offline*)	Limited to installed devices
Updates & Maintenance	Centralized; instant updates for all users	Each device must update manually (or via installer/patch)
Performance	Can be slower; depends on internet (*unless specifically designed offline*) & browser	Usually faster; takes full advantage of hardware
Offline Use	Limited (unless built as PWA or with caching) (*unless specifically designed offline*)	Works offline by default
Device Integration	Limited access to hardware (e.g., GPU, USB, Bluetooth)	Full access to system resources and hardware
Security	Centralized server risks, but easier to control	Security depends on local device; patching is harder
Scalability	Easy to scale; just add server resources	Scaling requires distributing and maintaining installs
User Experience (UI/UX)	May feel less responsive; browser limitations	Generally smoother and more responsive
Cross-Platform Support	Works on any device with a browser (Windows, Mac, Linux, mobile)	Usually OS-specific (Windows apps won't run on Mac without extra setup)
Cost of Deployment	Lower (single codebase, no installers)	Higher (need builds for different OS, distribution effort)

Project Timeline

Requirement Analysis – 2 weeks – Requirement document
Design – 2 weeks – Unified wireframes & UI mockups
Development – 12 weeks – Ticket + Inventory modules
Testing – 2 weeks – QA reports
Deployment & Training – 1 weeks – Live system + training docs

Total Estimated Duration: 21 weeks (5.25 months)

Cost Estimate

- **Requirement & Design**: PHP100,000
- **Development (Frontend & Backend)**: PHP480,000
- **Testing & QA**: PHP150,000
- **Deployment & Training**: PHP50,000
- **Support & Maintenance (2 months)**: PHP20,000

Total Estimated Cost: PHP800,000

Benefits

- Single platform for both ticket consolidation and inventory management.
- Reduce manual processes, ensuring **accuracy and compliance**.
- Provides near **real-time visibility** into ticket and stock expenses.
- Enhances reporting for finance and management.
- Scalable for additional modules in the future.

Next Steps

1. Approve or modify the expanded proposal and budget.
2. Finalize detailed requirements for both modules.
3. Kick off the design and development phase.

Contact Information

Prepared by: Mitch Quimpo / 3nnux Technologies Corporation
Email: mitch@trinnux.com
Phone: +639175028834
Date: September 15, 2025